

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO CUỐI KỲ
MÔN THỰC TẬP CƠ SỞ
MÃ HỌC PHẦN: INT13187

**Đề tài: Nghiên cứu, đánh giá và triển khai
mô hình Deep Learning tối ưu cho hệ thống phân loại rác thải**

Giảng viên hướng dẫn:	TS. Kim Ngọc Bách
Sinh viên thực hiện:	Nguyễn Văn Gia Bảo
Mã sinh viên:	B23DCCN079

Hà Nội - 2026

LỜI CẢM ƠN

Đầu tiên, em xin gửi lời cảm ơn sâu sắc đến Học viện Công nghệ Bưu chính Viễn thông và khoa CNTT1 đã đưa môn học Thực tập cơ sở vào trong chương trình giảng dạy. Đặc biệt, em xin gửi lời cảm ơn chân thành đến giảng viên bộ môn **TS. Kim Ngọc Bách** đã hướng dẫn chọn đề tài, phương pháp nghiên cứu và góp ý báo cáo, demo theo tiến độ hàng tuần cho em trong suốt thời gian học tập vừa qua.

Trong thời gian học tập, em đã được tiếp thu thêm nhiều kiến thức bổ ích, học tập được tinh thần làm việc hiệu quả, nghiêm túc. Đây thực sự là những điều rất cần thiết cho quá trình học tập và công tác sau này của em. Thêm vào đó, nhờ sự dẫn dắt và chỉ bảo của thầy, em đã thực hiện được dự án hoàn chỉnh cho môn học này, em rất biết ơn điều đó.

Em xin chân thành cảm ơn, chúc thầy luôn khỏe mạnh và tiếp tục đạt được nhiều thành công trong cuộc sống.

MỤC LỤC

LỜI CẢM ƠN	1
DANH MỤC HÌNH VẼ	1
DANH MỤC BẢNG BIỂU	2
DANH MỤC TỪ VIẾT TẮT.....	3
PHẦN MỞ ĐẦU	4
CHƯƠNG I. GIỚI THIỆU CHUNG.....	5
1.1 Đặt vấn đề.....	5
1.2. Mục tiêu đề tài	5
1.3. Phạm vi nghiên cứu	5
1.4. Phương pháp tiếp cận và định hướng giải pháp.....	5
1.4.1 Phương pháp tiếp cận	5
1.4.2 Định hướng giải pháp.....	6
1.5. Ý nghĩa khoa học và thực tiễn của đề tài	7
1.5.1 Ý nghĩa khoa học	7
1.5.2 Ý nghĩa thực tiễn.....	7
CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	8
2.1. Học sâu và mạng nơ-ron tích chập (CNN)	8
2.1.1 Tổng quan về học sâu	8
2.1.2 Mạng nơ-ron tích chập (CNN).....	8
2.1.3 Transfer Learning	8
2.2. Các kiến trúc mô hình sử dụng	8
2.2.1 VGG16	8
2.2.2 ResNet50.....	8
2.2.3 EfficientNet-B0	9
2.3. Các kỹ thuật huấn luyện nâng cao	9
2.3.1 Data Augmentation	9
2.3.2 Differential Learning Rate	9
2.3.3 Class Weighting	9
2.3.4 Adam Optimizer	9
2.4. Grad-CAM - Giải thích quyết định của mô hình	9
2.5. Gemini Vision AI	10
2.6. Công nghệ triển khai hệ thống	10
2.6.1 FastAPI và ONNX Runtime	10

2.6.2 Streamlit	10
2.6.3 SQLite3	10
CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	11
3.1. Phân tích yêu cầu	11
3.1.1 Yêu cầu chức năng	11
3.1.2 Yêu cầu phi chức năng	11
3.2. Bộ dữ liệu (dataset).....	11
3.2.1 Giai đoạn 1 - TrashNet (6 lớp)	11
3.2.2 Giai đoạn 2 - Garbage Classification (10 lớp)	11
3.3. Kiến trúc hệ thống.....	12
3.4. Thiết kế mô hình học sâu	12
3.4.1 Chiến lược so sánh mô hình (Giai đoạn 1).....	12
3.4.2 Chiến lược fine-tune EfficientNet-B0 (Giai đoạn 2)	13
3.4.3 Xuất mô hình ONNX.....	13
3.5. Thiết kế xử lý đặc biệt	13
3.5.1 Xử lý Grad-CAM an toàn.....	13
3.5.2 Cơ chế Fallback cho Gemini.....	13
3.5.3 Cảnh báo độ tin cậy thấp.....	13
3.6. Thiết kế cơ sở dữ liệu.....	13
3.7. Thiết kế chức năng xác thực người dùng	14
CHƯƠNG IV. KẾT QUẢ CÀI ĐẶT VÀ THỬ NGHIỆM.....	15
4.1. Môi trường thực nghiệm.....	15
4.2. Kết quả giai đoạn 1 - So sánh ba kiến trúc	15
4.3. Kết quả giai đoạn 2 - Fine-tune EfficientNet-B0	19
4.3.1 Hiệu năng tổng thể.....	19
4.3.2 Kết quả chi tiết theo từng lớp	19
4.3.3 Phân tích kết quả fine-tune.....	20
4.4. Kết quả triển khai ứng dụng.....	21
4.4.1 Chức năng đăng ký và đăng nhập.....	21
4.4.2 Chức năng phân loại	22
4.4.3 Hiển thị Grad-CAM.....	23
4.4.4 Tư vấn thông minh Gemini	24
4.4.5 Dashboard thống kê.....	24
4.4.6 Đánh giá tổng hợp.....	25

CHƯƠNG V. KẾT LUẬN.....	26
5.1. Tổng kết kết quả đạt được	26
5.2. Hạn chế.....	26
5.3. Hướng phát triển.....	26
5.4. Kết luận chung.....	27
TÀI LIỆU THAM KHẢO	28

DANH MỤC HÌNH VẼ

Hình 1 - Confusion matrix VGG16	16
Hình 2 - Confusion matrix ResNet50	17
Hình 3 - Confusion matrix EfficientNet-B0	18
Hình 4 – Confusion Matrix – EfficientNet Fine-tune.....	20
Hình 5 - Giao diện đăng ký thành công tài khoản	21
Hình 6 - Giao diện đăng nhập.....	21
Hình 7 - Giao diện phân loại (1)	22
Hình 8 - Giao diện phân loại (2)	22
Hình 9 - Giao diện phân loại (3)	23
Hình 10 - Giao diện phân loại (Grad-CAM).....	23
Hình 11 - Giao diện phân loại (Tur vắn thông minh Gemini)	24
Hình 12 - Giao diện Dashboard thống kê (1).....	24
Hình 13 - Giao diện Dashboard thống kê (2).....	25
Hình 14 - Giao diện Dashboard thống kê (3).....	25

DANH MỤC BẢNG BIỂU

Bảng 1 - VGG16 classification report.....	15
Bảng 2 - ResNet50 classification report.....	16
Bảng 3 - EfficientNet-B0 classification report.....	17
Bảng 4 - Kết quả so sánh ba mô hình	18
Bảng 5 - Kết quả đánh giá sau khi train EfficientNet-B0	19

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
ONNX	Open Neural Network Exchange	Định dạng trao đổi mô hình mạng nơ-ron mở
GPU	Graphics Processing Unit	Bộ xử lý đồ họa

PHẦN MỞ ĐẦU

Rác thải là một trong những vấn đề môi trường cấp bách nhất hiện nay. Mỗi năm toàn thế giới thải ra hơn 2 tỷ tấn rác, trong đó tỷ lệ tái chế còn rất thấp. Tại Việt Nam, ý thức phân loại rác tại nguồn chưa cao, cộng với thiếu công cụ hỗ trợ để giúp mọi người phân loại rác một cách nhanh chóng và chính xác.

Phân loại rác thủ công đòi hỏi nhiều nhân lực, tốn thời gian và dễ sai sót. Trong bối cảnh công nghệ trí tuệ nhân tạo (AI) và học sâu (Deep Learning) phát triển mạnh mẽ, việc áp dụng các mô hình nhận dạng hình ảnh vào phân loại rác thải tự động là một hướng tiếp cận đầy triển vọng.

Xuất phát từ những nhu cầu đó, em đã lựa chọn đề tài: **"Nghiên cứu, Đánh giá và Triển khai Mô hình Deep Learning Tối ưu cho Hệ thống Phân loại Rác thải"**. Hệ thống được xây dựng bằng Python, sử dụng PyTorch cho huấn luyện mô hình, FastAPI cho backend và Streamlit cho frontend. Mục tiêu là tạo ra một ứng dụng giúp người dùng phân loại rác dễ dàng, kèm theo giải thích quyết định (Grad-CAM) và tư vấn thông minh (Gemini Vision).

Báo cáo gồm 5 chương chính:

Chương 1: Giới thiệu chung - Nêu vấn đề, mục tiêu và phạm vi đề tài.

Chương 2: Cơ sở lý thuyết và công nghệ sử dụng - Giới thiệu học sâu, CNN, transfer learning, các kiến trúc mô hình (VGG16, ResNet50, EfficientNet-B0), Grad-CAM và các framework triển khai.

Chương 3: Phân tích và thiết kế hệ thống - Mô tả yêu cầu hệ thống, bộ dữ liệu, kiến trúc client-server, thiết kế mô hình và cơ sở dữ liệu.

Chương 4: Kết quả cài đặt và thử nghiệm - Trình bày kết quả so sánh ba mô hình, fine-tune mô hình tối ưu, các tính năng ứng dụng và đánh giá tổng hợp.

Chương 5: Kết luận - Tóm tắt kết quả, phân tích hạn chế và hướng phát triển tiếp theo.

CHƯƠNG I. GIỚI THIỆU CHUNG

1.1 Đặt vấn đề

- Rác thải là một trong những vấn đề môi trường cấp bách nhất hiện nay. Theo thống kê, mỗi năm thế giới thải ra hơn 2 tỷ tấn rác, trong đó chỉ một phần nhỏ được phân loại và tái chế đúng cách. Tại Việt Nam, tỷ lệ tái chế rác thải còn rất thấp, một phần do ý thức phân loại rác tại nguồn chưa cao, một phần do thiếu công cụ hỗ trợ người dân nhận biết và phân loại rác một cách chính xác.

- Phân loại rác thủ công đòi hỏi nhiều nhân lực, tốn thời gian và dễ xảy ra sai sót. Trong bối cảnh trí tuệ nhân tạo (AI) và học sâu (Deep Learning) đang phát triển mạnh mẽ, việc áp dụng các mô hình nhận dạng hình ảnh vào bài toán phân loại rác thải tự động là một hướng tiếp cận đầy tiềm năng.

1.2. Mục tiêu đề tài

- Đề tài tập trung vào ba mục tiêu chính:

- Thứ nhất, nghiên cứu và so sánh hiệu năng của ba kiến trúc mạng nơ-ron tích chập phổ biến là VGG16, ResNet50 và EfficientNet-B0 trên bài toán phân loại rác thải, từ đó lựa chọn kiến trúc tối ưu nhất.
- Thứ hai, tiến hành fine-tune chuyên sâu mô hình được chọn trên bộ dữ liệu rác thải 10 lớp, áp dụng các kỹ thuật hiện đại như differential learning rate, class weighting và data augmentation để đạt độ chính xác cao nhất có thể.
- Thứ ba, xây dựng và triển khai một ứng dụng web hoàn chỉnh tích hợp mô hình AI, kèm theo khả năng giải thích quyết định của mô hình thông qua Grad-CAM và tư vấn phân loại thông minh bằng Gemini Vision AI.

1.3. Phạm vi nghiên cứu

- Đề tài được thực hiện bao gồm hai giai đoạn.

- Giai đoạn 1 sử dụng bộ dữ liệu TrashNet với 6 lớp để so sánh ba kiến trúc mô hình.
- Giai đoạn 2 sử dụng bộ dữ liệu Garbage Classification với 10 lớp để fine-tune mô hình tối ưu và triển khai ứng dụng thực tế.

1.4. Phương pháp tiếp cận và định hướng giải pháp

1.4.1 Phương pháp tiếp cận

- Để giải quyết bài toán phân loại rác thải từ hình ảnh, đề tài tiếp cận theo hướng ứng dụng học sâu (Deep Learning) trong lĩnh vực thị giác máy tính (Computer Vision). Thay vì sử dụng các phương pháp trích xuất đặc trưng thủ công truyền thống, hệ thống sử dụng các mạng nơ-ron tích chập (CNN) để tự động học các đặc trưng quan trọng từ dữ liệu ảnh rác thải.

- Quá trình thực hiện được tiến hành theo các bước chính sau:

- Nghiên cứu các kiến trúc CNN phổ biến cho bài toán phân loại ảnh như VGG16, ResNet50 và EfficientNet-B0.
 - Thực hiện huấn luyện và so sánh hiệu năng giữa các mô hình trên cùng bộ dữ liệu TrashNet nhằm lựa chọn kiến trúc phù hợp nhất.
 - Áp dụng kỹ thuật Transfer Learning để tận dụng trọng số đã được huấn luyện trước trên ImageNet.
 - Thực hiện fine-tune mô hình EfficientNet-B0 trên bộ dữ liệu Garbage Classification 10 lớp.
 - Áp dụng các kỹ thuật cải thiện hiệu năng như Data Augmentation, Differential Learning Rate và Class Weighting.
 - Triển khai mô hình dưới dạng ONNX để tối ưu tốc độ suy luận.
 - Xây dựng hệ thống backend bằng FastAPI và frontend bằng Streamlit.
 - Tích hợp Grad-CAM nhằm giải thích quyết định của mô hình và Gemini Vision AI nhằm hỗ trợ tư vấn phân loại rác thông minh.
- Phương pháp tiếp cận này giúp kết hợp giữa độ chính xác của mô hình học sâu và khả năng triển khai thực tế của hệ thống.

1.4.2 Định hướng giải pháp

- Đề tài lựa chọn giải pháp xây dựng hệ thống phân loại rác theo mô hình client-server hai tầng.
- Trong đó:
- Backend sử dụng FastAPI kết hợp ONNX Runtime để thực hiện suy luận mô hình với tốc độ cao.
 - Frontend sử dụng Streamlit để xây dựng giao diện tương tác với người dùng.
 - EfficientNet-B0 được lựa chọn làm mô hình chính do có sự cân bằng tốt giữa độ chính xác và kích thước mô hình.
 - SQLite3 được sử dụng để lưu trữ thông tin người dùng, lịch sử phân loại và phản hồi hệ thống.
 - Grad-CAM được sử dụng để trực quan hóa vùng ảnh mà mô hình tập trung khi đưa ra dự đoán.
 - Gemini Vision AI đóng vai trò như một lớp hỗ trợ tư vấn thông minh nhằm tăng tính thực tiễn của hệ thống.
- Giải pháp được xây dựng hướng tới mục tiêu đạt độ chính xác cao, tốc độ phản hồi nhanh, dễ triển khai và có khả năng mở rộng trong tương lai.

1.5. Ý nghĩa khoa học và thực tiễn của đề tài

1.5.1 Ý nghĩa khoa học

- Đề tài góp phần nghiên cứu và đánh giá hiệu quả của các mô hình học sâu hiện đại trong bài toán phân loại rác thải từ hình ảnh. Thông qua việc so sánh các kiến trúc CNN phổ biến như VGG16, ResNet50 và EfficientNet-B0, đề tài giúp làm rõ ưu điểm và hạn chế của từng mô hình trong điều kiện dữ liệu thực tế.
- Bên cạnh đó, đề tài còn nghiên cứu khả năng áp dụng các kỹ thuật hiện đại như Transfer Learning, Data Augmentation, Differential Learning Rate và Class Weighting nhằm cải thiện độ chính xác mô hình.
- Ngoài ra, việc tích hợp Grad-CAM giúp tăng khả năng giải thích quyết định của mô hình AI, góp phần nâng cao tính minh bạch trong các hệ thống học sâu.
- Đề tài cũng thử nghiệm việc kết hợp mô hình phân loại ảnh với Gemini Vision AI nhằm mở rộng khả năng hỗ trợ tư vấn thông minh cho người dùng.

1.5.2 Ý nghĩa thực tiễn

- Hệ thống phân loại rác được xây dựng có khả năng hỗ trợ người dùng nhận biết và phân loại rác nhanh chóng thông qua hình ảnh, góp phần nâng cao ý thức bảo vệ môi trường và thúc đẩy hoạt động tái chế.
- Ngoài ra, hệ thống có thể được ứng dụng trong:
 - Hỗ trợ phân loại rác tại nguồn trong hộ gia đình.
 - Hệ thống giáo dục và tuyên truyền bảo vệ môi trường.
 - Các hệ thống thùng rác thông minh trong mô hình Smart City.
 - Hỗ trợ doanh nghiệp tái chế trong quá trình phân loại sơ bộ.
- Việc triển khai hệ thống dưới dạng web application giúp dễ dàng mở rộng và tích hợp vào các nền tảng thực tế trong tương lai.

CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Học sâu và mạng nơ-ron tích chập (CNN)

2.1.1 Tổng quan về học sâu

- Học sâu (Deep Learning) là một nhánh của học máy (Machine Learning), sử dụng các mạng nơ-ron nhân tạo nhiều tầng để học các đặc trưng phức tạp từ dữ liệu thô [1]. Khác với các phương pháp truyền thống yêu cầu trích xuất đặc trưng thủ công, học sâu tự động học được các biểu diễn phân cấp từ dữ liệu đầu vào, từ các đặc trưng cấp thấp như cạnh, góc đến các đặc trưng cấp cao như hình dạng, kết cấu bề mặt và ngữ nghĩa.

2.1.2 Mạng nơ-ron tích chập (CNN)

- Mạng nơ-ron tích chập (Convolutional Neural Network - CNN) là kiến trúc học sâu đặc biệt hiệu quả cho bài toán xử lý ảnh [1]. CNN khai thác tính chất không gian của ảnh thông qua ba loại tầng chính. Tầng tích chập (Convolutional Layer) áp dụng các bộ lọc (filter) trượt qua ảnh đầu vào để trích xuất đặc trưng cục bộ. Tầng gộp (Pooling Layer) giảm chiều không gian của đặc trưng, giúp mô hình bất biến với các phép biến đổi nhỏ như dịch chuyển hay xoay. Tầng kết nối đầy đủ (Fully Connected Layer) tổng hợp các đặc trưng đã học để đưa ra dự đoán cuối cùng.

2.1.3 Transfer Learning

- Transfer Learning là kỹ thuật tận dụng tri thức đã học của một mô hình được huấn luyện trên tập dữ liệu lớn (thường là ImageNet với 1.2 triệu ảnh, 1000 lớp) [6] để giải quyết bài toán mới với dữ liệu ít hơn. Thay vì huấn luyện từ đầu, ta chỉ cần thay thế lớp phân loại cuối cùng và fine-tune mô hình trên tập dữ liệu đích. Cách tiếp cận này giúp tiết kiệm đáng kể thời gian và tài nguyên tính toán, đồng thời đạt được độ chính xác cao ngay cả với tập dữ liệu nhỏ.

2.2. Các kiến trúc mô hình sử dụng

2.2.1 VGG16

- VGG16 được giới thiệu bởi Simonyan và Zisserman năm 2015 [2], có kiến trúc đơn giản với 16 tầng có trọng số, sử dụng các bộ lọc tích chập 3×3 đồng nhất xuyên suốt mạng. Ưu điểm của VGG16 là kiến trúc rõ ràng, dễ hiểu, và đã được chứng minh hiệu quả trên nhiều bài toán nhận dạng ảnh. Nhược điểm lớn nhất là kích thước mô hình rất lớn, khoảng 512 MB, chủ yếu do ba lớp Fully Connected với số lượng tham số rất lớn.

2.2.2 ResNet50

- ResNet50 (Residual Network 50 layers) được He và cộng sự giới thiệu năm 2016 [3], giải quyết vấn đề gradient vanishing trong mạng sâu thông qua kết nối tắt (skip connection). Kết nối tắt cho phép gradient truyền ngược trực tiếp qua nhiều tầng, giúp huấn luyện các mạng rất sâu hiệu quả hơn. ResNet50 có khoảng 25 triệu tham số với kích thước file khoảng 90 MB, cân bằng tốt giữa độ chính xác và tài nguyên.

2.2.3 EfficientNet-B0

- EfficientNet được Tan và Le đề xuất năm 2019 [4] với phương pháp mở rộng mô hình theo cách cân bằng (compound scaling), đồng thời tăng độ sâu (depth), độ rộng (width) và độ phân giải (resolution) của mạng theo một hệ số thống nhất. EfficientNet-B0 là phiên bản nhỏ nhất trong họ EfficientNet, chỉ có khoảng 5.3 triệu tham số với kích thước file 15.6 MB, nhưng đạt độ chính xác tương đương hoặc vượt trội so với các kiến trúc lớn hơn nhiều lần. Đây là lý do EfficientNet-B0 được lựa chọn để triển khai thực tế trong đề tài này.

2.3. Các kỹ thuật huấn luyện nâng cao

2.3.1 Data Augmentation

- Data Augmentation là kỹ thuật tăng cường dữ liệu bằng cách tạo ra các biến thể của ảnh gốc thông qua các phép biến đổi như lật ngang (horizontal flip), cắt ngẫu nhiên (random crop), xoay (rotation) và thay đổi màu sắc (color jitter). Kỹ thuật này giúp tăng tính đa dạng của tập huấn luyện, giảm hiện tượng overfitting và cải thiện khả năng tổng quát hóa của mô hình.

2.3.2 Differential Learning Rate

- Differential Learning Rate là kỹ thuật sử dụng các tốc độ học khác nhau cho các phần khác nhau của mạng trong quá trình fine-tune. Cụ thể, các tầng backbone được sử dụng với learning rate rất nhỏ ($1e-5$) để giữ lại tri thức đã học từ ImageNet [6], trong khi lớp classifier mới được học với learning rate lớn hơn ($1e-4$) để nhanh chóng thích nghi với nhiệm vụ mới.

2.3.3 Class Weighting

- Trong bộ dữ liệu thực tế, số lượng mẫu giữa các lớp thường không đồng đều (class imbalance). Để xử lý vấn đề này, trọng số lớp (class weight) tỉ lệ nghịch với số lượng mẫu của lớp đó được áp dụng vào hàm mất mát. Lớp có ít mẫu hơn sẽ nhận trọng số cao hơn, buộc mô hình phải chú ý nhiều hơn đến các lớp thiểu số trong quá trình học.

2.3.4 Adam Optimizer

- Adam (Adaptive Moment Estimation) là thuật toán tối ưu hóa được Kingma và Ba giới thiệu năm 2015 [15], kết hợp ưu điểm của hai phương pháp AdaGrad và RMSProp. Adam tự động điều chỉnh learning rate cho từng tham số dựa trên moment bậc nhất và bậc hai của gradient, giúp hội tụ nhanh và ổn định hơn so với SGD thuần túy.

2.4. Grad-CAM - Giải thích quyết định của mô hình

- Gradient-weighted Class Activation Mapping (Grad-CAM) là phương pháp trực quan hóa được Selvaraju và cộng sự đề xuất năm 2017 [5], giúp giải thích quyết định của CNN bằng cách hiển thị vùng nào trên ảnh đầu vào có ảnh hưởng lớn nhất đến kết quả phân loại. Grad-CAM tính gradient của điểm số lớp được dự đoán đối với các bản đồ đặc trưng (feature maps) của tầng tích chập cuối cùng, sau đó kết hợp có trọng số để tạo

ra bản đồ nhiệt (heatmap). Trong đề tài, Grad-CAM sử dụng tầng features[1] của EfficientNet-B0 và được triển khai thông qua thư viện pytorch-grad-cam.

2.5. Gemini Vision AI

- Gemini Vision là mô hình đa phương thức của Google, có khả năng phân tích và hiểu nội dung ảnh kết hợp với văn bản [12]. Trong hệ thống GreenAI, Gemini Vision (model gemini-3.5-flash) được sử dụng như một lớp tư vấn thông minh, bổ sung cho kết quả phân loại của mô hình deep learning. Cụ thể, Gemini phân tích tình trạng vật thể trong ảnh và xác định khả năng tái chế (recyclable, non_recyclable, hazardous), từ đó kết hợp với nhãn lớp để đưa ra lời khuyên phân loại phù hợp.

2.6. Công nghệ triển khai hệ thống

2.6.1 FastAPI và ONNX Runtime

- FastAPI là framework Python hiện đại để xây dựng REST API, nổi bật với hiệu năng cao và hỗ trợ bất đồng bộ (async) [9]. ONNX (Open Neural Network Exchange) là định dạng mô hình mở cho phép chạy mô hình trên nhiều nền tảng và framework khác nhau. ONNX Runtime là công cụ inference hiệu năng cao cho mô hình ONNX [11], nhanh hơn đáng kể so với PyTorch thuần trong giai đoạn suy luận. Kết hợp FastAPI và ONNX Runtime cho phép xây dựng một backend phân loại ảnh nhanh, nhẹ và dễ triển khai.

2.6.2 Streamlit

- Streamlit là thư viện Python cho phép xây dựng ứng dụng web tương tác nhanh chóng mà không cần kiến thức sâu về HTML/CSS/JavaScript [10]. Đây là lựa chọn phù hợp cho việc phát triển giao diện người dùng trong các dự án AI/ML, giúp tập trung vào logic ứng dụng hơn là thiết kế giao diện.

2.6.3 SQLite3

- SQLite3 là hệ quản trị cơ sở dữ liệu quan hệ nhúng (embedded), không cần server riêng, phù hợp với các ứng dụng quy mô vừa và nhỏ. Hệ thống sử dụng SQLite3 để lưu trữ thông tin người dùng, lịch sử phân loại và dữ liệu phản hồi.

CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

3.1.1 Yêu cầu chức năng

- Hệ thống cần đáp ứng các yêu cầu chức năng sau. Về phía người dùng, hệ thống phải cho phép đăng ký và đăng nhập tài khoản, upload ảnh rác thải để phân loại, xem kết quả phân loại kèm độ tin cậy top-3, xem bản đồ nhiệt Grad-CAM [5] để hiểu mô hình đang chú ý vào vùng nào, nhận tư vấn thông minh về cách xử lý rác, cung cấp phản hồi về kết quả phân loại, và theo dõi lịch sử phân loại qua dashboard thống kê.
- Về phía hệ thống, backend phải thực hiện inference nhanh chóng bằng ONNX Runtime [11], tự động sao lưu cơ sở dữ liệu hàng ngày, hoạt động ổn định ngay cả khi mất kết nối đến Gemini API, và lưu trữ ảnh lịch sử một cách có tổ chức.

3.1.2 Yêu cầu phi chức năng

- Về hiệu năng, thời gian phản hồi cho một lần phân loại phải đủ nhanh để không ảnh hưởng trải nghiệm người dùng. Về độ chính xác, mô hình phải đạt tối thiểu 90% test accuracy trên tập dữ liệu 10 lớp. Về khả năng giải thích, hệ thống phải cung cấp trực quan hóa Grad-CAM rõ ràng. Về độ tin cậy, hệ thống phải có cơ chế fallback khi các dịch vụ bên ngoài không khả dụng.

3.2. Bộ dữ liệu (dataset)

3.2.1 Giai đoạn 1 - TrashNet (6 lớp)

- Bộ dữ liệu TrashNet được thu thập bởi Gary Thung và cộng sự [7], gồm 6 lớp: cardboard, glass, metal, paper, plastic và trash. Bộ dữ liệu này được sử dụng trong giai đoạn so sánh ba kiến trúc mô hình với điều kiện dữ liệu và cấu hình huấn luyện thống nhất.

3.2.2 Giai đoạn 2 - Garbage Classification (10 lớp)

- Bộ dữ liệu Garbage Classification từ Kaggle [8] có 11 lớp gốc. Để chuẩn bị dữ liệu, hai lớp biological và organic được gộp lại thành một lớp organic do nội dung tương tự nhau. Một số ảnh được loại bỏ để cân bằng dữ liệu. Sau xử lý, bộ dữ liệu gồm 4.462 ảnh thuộc 10 lớp: battery (504), cardboard (403), clothes (504), glass (501), metal (410), organic (513), paper (504), plastic (482), shoes (504) và trash (137).
- Dữ liệu được chia theo tỉ lệ 70/15/15 (train/val/test) với phương pháp stratified sampling để đảm bảo phân phối lớp đồng đều, sử dụng random_state=42 để đảm bảo khả năng tái hiện kết quả. Kết quả là 3.123 ảnh train, 669 ảnh validation và 670 ảnh test.
- Đáng chú ý là lớp trash chỉ có 137 ảnh, ít nhất trong tất cả các lớp, dẫn đến class weight cao nhất là 3.2531. Đây là điểm yếu khó tránh khỏi của bộ dữ liệu và ảnh hưởng trực tiếp đến hiệu năng phân loại của lớp này.

3.2.3 Tiền xử lý dữ liệu ảnh

- Trước khi đưa vào huấn luyện, dữ liệu ảnh được thực hiện tiền xử lý nhằm đảm bảo tính đồng nhất và tăng hiệu quả học của mô hình.

Các bước tiền xử lý bao gồm:

- Resize toàn bộ ảnh về kích thước chuẩn 224×224 pixel phù hợp với đầu vào của các mô hình CNN pretrained trên ImageNet.
- Chuyển đổi ảnh sang không gian màu RGB.
- Chuẩn hóa dữ liệu ảnh theo mean và standard deviation của ImageNet nhằm đảm bảo dữ liệu đầu vào có phân phối phù hợp với mô hình pretrained.
- Chuyển đổi ảnh sang Tensor để phục vụ quá trình huấn luyện bằng PyTorch.

- Ngoài ra, trong quá trình huấn luyện, đề tài áp dụng các kỹ thuật Data Augmentation nhằm tăng tính đa dạng của dữ liệu:

- Random Horizontal Flip
- Random Rotation
- Random Resized Crop
- Color Jitter

- Các kỹ thuật này giúp giảm hiện tượng overfitting và cải thiện khả năng tổng quát hóa của mô hình trên dữ liệu thực tế.

3.3. Kiến trúc hệ thống

- Hệ thống GreenAI được tổ chức theo mô hình client-server hai tầng rõ ràng. Backend là một REST API được xây dựng bằng FastAPI [9], chạy trên cổng 8000, chịu trách nhiệm toàn bộ logic xử lý: inference mô hình bằng ONNX Runtime [11], tính toán Grad-CAM [5] và phản hồi kết quả dưới dạng JSON. Frontend là ứng dụng Streamlit [10] chạy trên cổng 8501, chịu trách nhiệm toàn bộ giao diện người dùng, giao tiếp với backend qua HTTP POST đến endpoint /predict và giao tiếp với Gemini API [12] để lấy tư vấn.

- Luồng xử lý khi người dùng upload ảnh diễn ra như sau. Người dùng tải ảnh lên giao diện Streamlit. Frontend gửi ảnh qua HTTP POST đến api.py. Backend chạy ONNX Runtime để lấy top-3 predictions kèm confidence. Song song, backend chạy Grad-CAM bằng PyTorch model để tạo heatmap và mã hóa dạng base64. Backend trả về JSON chứa predictions và heatmap cho frontend. Frontend gọi Gemini Vision để lấy tư vấn phân loại. Kết quả được lưu vào SQLite và ảnh gốc được lưu vào thư mục history_images. Cuối cùng, toàn bộ kết quả được hiển thị lên giao diện người dùng.

3.4. Thiết kế mô hình học sâu

3.4.1 Chiến lược so sánh mô hình (Giai đoạn 1)

- Ba mô hình được huấn luyện với cấu hình chuẩn phổ biến của từng kiến trúc. VGG16 [2] mở đóng bằng toàn bộ classifier gồm 5 lớp Fully Connected (~119M tham số có thể học). ResNet50 [3] đóng bằng toàn bộ mạng và chỉ thay thế lớp fc cuối (~10K tham số).

EfficientNet-B0 [4] đóng băng toàn bộ và chỉ thay thế classifier[1] (~7K tham số). Tất cả đều sử dụng Adam optimizer [15] với $lr=0.001$, StepLR scheduler, CrossEntropyLoss và huấn luyện 10 epoch trên 6 lớp.

3.4.2 Chiến lược fine-tune EfficientNet-B0 (Giai đoạn 2)

- Sau khi EfficientNet-B0 [4] được chọn, toàn bộ backbone được mở đóng băng để fine-tune, kết hợp với differential learning rate: backbone sử dụng $lr=1e-5$ để bảo toàn tri thức đã học từ ImageNet [6], trong khi classifier mới sử dụng $lr=1e-4$ để học nhanh hơn trên nhiệm vụ phân loại rác 10 lớp. Hàm mất mát CrossEntropyLoss được tích hợp class weights để bù cho sự mất cân bằng dữ liệu. Scheduler ReduceLROnPlateau tự động giảm learning rate khi val loss ngừng cải thiện sau 5 epoch liên tiếp. Mô hình được huấn luyện trong 40 epoch, lưu checkpoint theo best validation accuracy.

3.4.3 Xuất mô hình ONNX

- Sau khi huấn luyện, mô hình PyTorch (.pth) được xuất sang định dạng ONNX (.onnx) [11] với opset_version=11 và hỗ trợ dynamic_axes cho batch_size. Trong quá trình inference thực tế, ONNX Runtime được dùng để chạy mô hình, giúp tốc độ inference nhanh hơn so với PyTorch. Riêng Grad-CAM [5] vẫn sử dụng mô hình PyTorch gốc vì cần truy cập gradient.

3.5. Thiết kế xử lý đặc biệt

3.5.1 Xử lý Grad-CAM an toàn

- Mỗi yêu cầu Grad-CAM sử dụng file tạm với UUID ngẫu nhiên để tránh xung đột khi nhiều yêu cầu đến đồng thời (race condition). File tạm được dọn sạch trong khối finally để đảm bảo không bị rò rỉ. Nếu Grad-CAM gặp lỗi, hệ thống không crash mà chỉ bỏ qua phần heatmap, kết quả phân loại chính vẫn được trả về bình thường.

3.5.2 Cơ chế Fallback cho Gemini

- Khi không có kết nối mạng hoặc Gemini API [12] không khả dụng, hệ thống sử dụng một từ điển fallback được định nghĩa sẵn, ánh xạ từng lớp rác với lời khuyên xử lý mặc định. Cơ chế này đảm bảo ứng dụng luôn hoạt động được ngay cả trong môi trường offline.

3.5.3 Cảnh báo độ tin cậy thấp

- Khi confidence của dự đoán hàng đầu thấp hơn 60%, giao diện hiển thị cảnh báo về độ tin cậy thấp để người dùng biết kết quả có thể không chắc chắn, tránh hiểu lầm hoặc sử dụng kết quả không chính xác.

3.6. Thiết kế cơ sở dữ liệu

- Cơ sở dữ liệu SQLite3 gồm ba bảng. Bảng users lưu thông tin tài khoản với mật khẩu được mã hóa bằng bcrypt. Bảng history lưu mỗi lần phân loại gồm username, thời gian, loại rác, độ tin cậy, danh mục và đường dẫn ảnh. Bảng feedback lưu phản hồi của người dùng về kết quả phân loại, bao gồm lớp được dự đoán, lớp thực tế và đánh giá đúng/sai.

Dữ liệu feedback này tạo nền tảng cho việc retrain mô hình trong tương lai theo hướng Active Learning. Hệ thống tự động sao lưu cơ sở dữ liệu một lần mỗi ngày vào thư mục backup_db.

3.7. Thiết kế chức năng xác thực người dùng

- Hệ thống GreenAI yêu cầu người dùng đăng ký và đăng nhập trước khi sử dụng, nhằm đảm bảo mỗi lịch sử phân loại và phản hồi được gắn với tài khoản cụ thể, phục vụ cho việc thống kê và thu thập dữ liệu feedback về sau.
- Chức năng xác thực được triển khai trong app.py và database.py. Khi đăng ký, hệ thống kiểm tra username chưa tồn tại trong bảng users, sau đó mã hóa mật khẩu bằng thư viện bcrypt trước khi lưu vào cơ sở dữ liệu SQLite3 - đảm bảo mật khẩu không bao giờ được lưu dưới dạng văn bản thuần. Khi đăng nhập, hệ thống truy vấn bảng users, lấy hash đã lưu và dùng bcrypt để so sánh với mật khẩu người dùng nhập vào. Nếu khớp, thông tin phiên đăng nhập được lưu vào st.session_state để duy trì trạng thái xuyên suốt các trang của ứng dụng Streamlit.
- Thiết kế này đảm bảo hai yêu cầu bảo mật cơ bản: mật khẩu được mã hóa một chiều (không thể giải mã ngược), và mỗi người dùng chỉ truy cập được lịch sử của chính mình.

CHƯƠNG IV. KẾT QUẢ CÀI ĐẶT VÀ THỬ NGHIỆM

4.1. Môi trường thực nghiệm

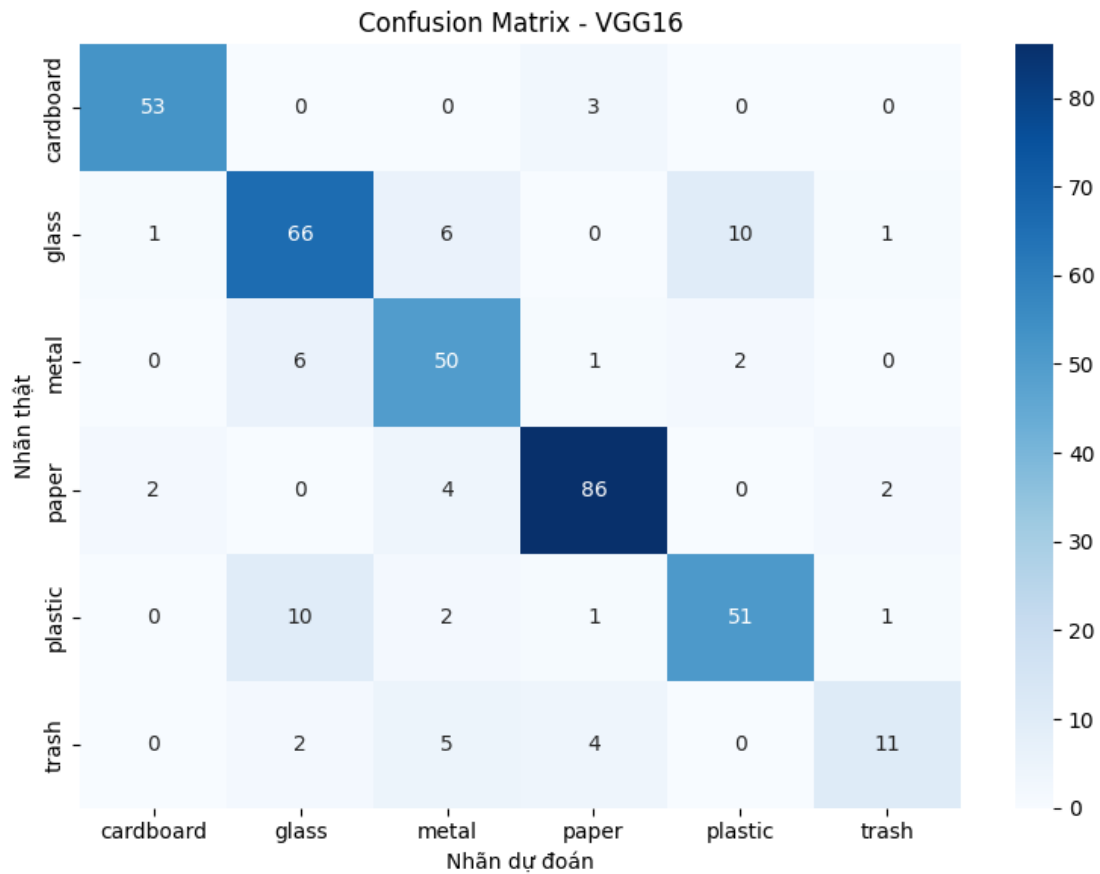
- Hệ thống được phát triển và thử nghiệm trên môi trường Python với các thư viện chính gồm PyTorch và TorchVision cho huấn luyện mô hình, ONNX Runtime [11] cho inference, FastAPI [9] và Uvicorn cho backend, Streamlit [10] cho frontend, pytorch-grad-cam cho Grad-CAM [5], google-genai cho tích hợp Gemini [12], và các thư viện hỗ trợ như OpenCV, scikit-learn, Plotly, Pandas và Bcrypt.

4.2. Kết quả giai đoạn 1 - So sánh ba kiến trúc

- VGG16 Test Accuracy: 83.42%

Class	Precision	Recall	F1-score	Support
cardboard	0.95	0.95	0.95	56
glass	0.79	0.79	0.79	84
metal	0.75	0.85	0.79	59
paper	0.91	0.91	0.91	94
plastic	0.81	0.78	0.80	65
trash	0.73	0.50	0.59	22
Accuracy			0.83	380
Macro Avg	0.82	0.80	0.80	380
Weighted Avg	0.83	0.83	0.83	380

Bảng 1 - VGG16 classification report

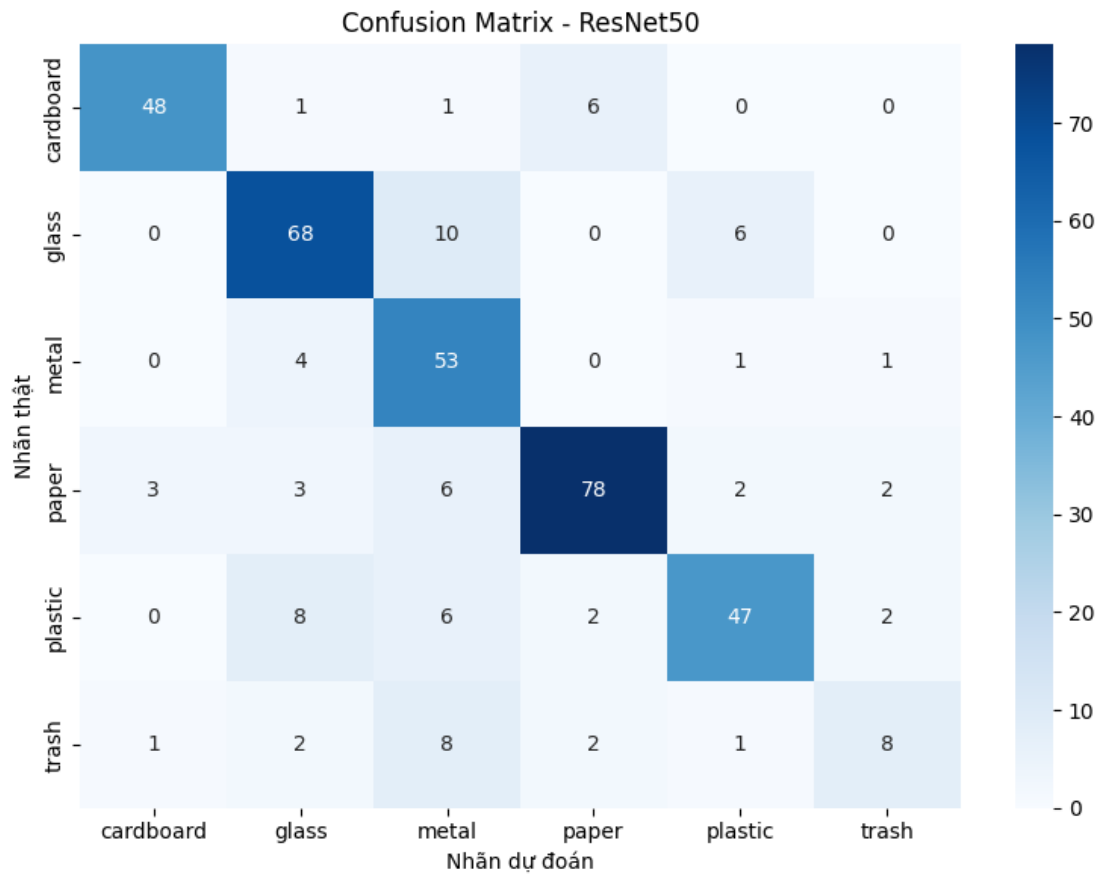


Hình 1 - Confusion matrix VGG16

- ResNet50 Test Accuracy: 79.47%

Class	Precision	Recall	F1-score	Support
cardboard	0.92	0.86	0.89	56
glass	0.79	0.81	0.80	84
metal	0.63	0.90	0.74	59
paper	0.89	0.83	0.86	94
plastic	0.82	0.72	0.77	65
trash	0.62	0.36	0.46	22
Accuracy			0.79	380
Macro Avg	0.78	0.75	0.75	380
Weighted Avg	0.80	0.79	0.79	380

Bảng 2 - ResNet50 classification report

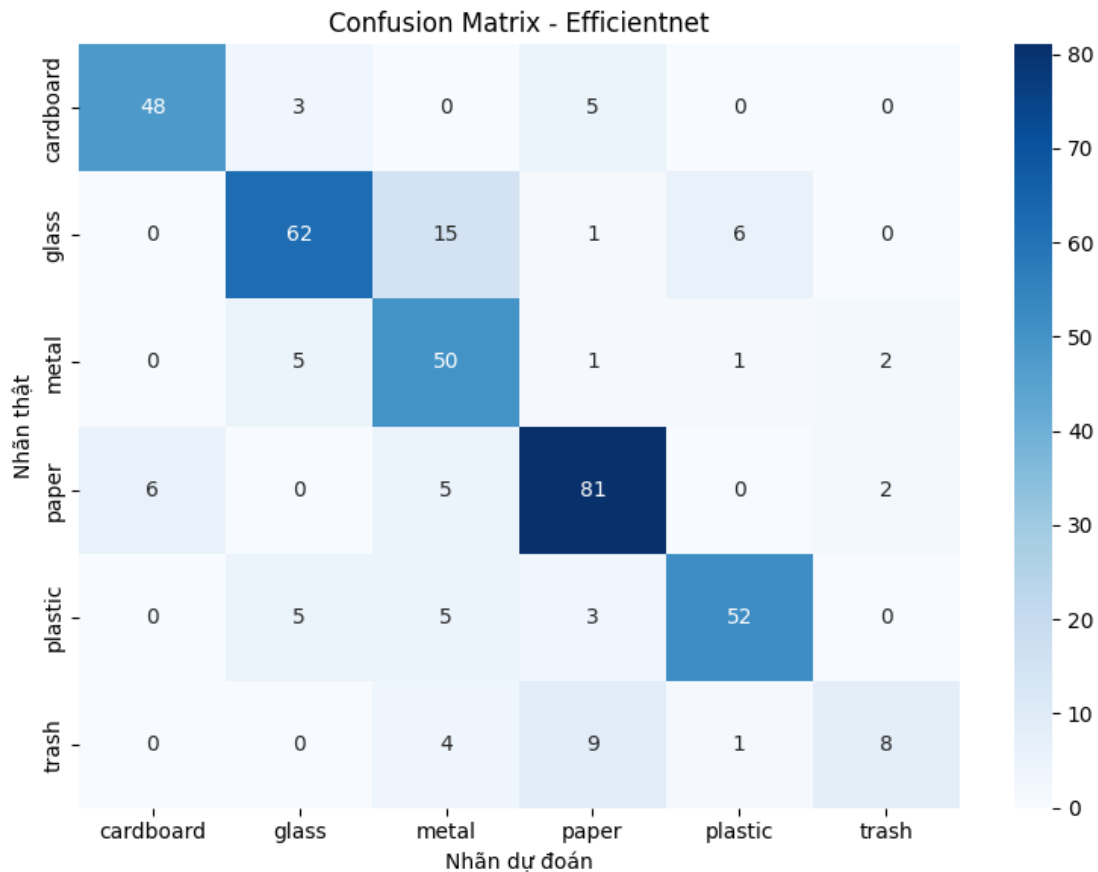


Hình 2 - Confusion matrix ResNet50

- EfficientNet-B0 Test Accuracy: 79.21%

Class	Precision	Recall	F1-score	Support
cardboard	0.89	0.86	0.87	56
glass	0.83	0.74	0.78	84
metal	0.63	0.85	0.72	59
paper	0.81	0.86	0.84	94
plastic	0.87	0.80	0.83	65
trash	0.67	0.36	0.47	22
Accuracy			0.79	380
Macro Avg	0.78	0.74	0.75	380
Weighted Avg	0.80	0.79	0.79	380

Bảng 3 - EfficientNet-B0 classification report



Hình 3 - Confusion matrix EfficientNet-B0

- Ba mô hình được huấn luyện và đánh giá trên cùng bộ dữ liệu TrashNet 6 lớp [7]. Kết quả như sau.

Tiêu chí	VGG16	ResNet50	EfficientNet-B0
Test Accuracy	83.42%	79.47%	79.21%
Macro F1-score	0.80	0.75	0.75
Dung lượng tệp	~512 MB	~90 MB	~15.6 MB
Số lớp Fine-tune	6 (Phần Classifier)	2 (Lớp Linear cuối)	2 (Lớp Linear cuối)
Tốc độ huấn luyện	Chậm nhất	Trung bình	Nhanh nhất

Bảng 4 - Kết quả so sánh ba mô hình

- VGG16 [2] đạt test accuracy cao nhất là 83.42% với macro F1-score 0.80, nhưng có kích thước mô hình rất lớn (~512 MB) và tốc độ huấn luyện rất chậm. ResNet50 [3] đạt test accuracy 79.47% với macro F1-score 0.75, kích thước ~90 MB, tốc độ huấn luyện trung bình. EfficientNet-B0 [4] đạt test accuracy 79.21% với macro F1-score 0.75, kích thước chỉ ~15.6 MB và tốc độ huấn luyện nhanh nhất.

- Mặc dù VGG16 có accuracy cao hơn, EfficientNet-B0 được chọn để tiến hành fine-tune chuyên sâu vì ba lý do.

- Thứ nhất, kích thước mô hình nhỏ hơn VGG16 đến 33 lần, phù hợp hơn cho triển khai thực tế.
- Thứ hai, tốc độ huấn luyện và inference nhanh hơn đáng kể.
- Thứ ba, khoảng cách accuracy so với VGG16 (4.21%) hoàn toàn có thể thu hẹp được qua fine-tune chuyên sâu trên bộ dữ liệu lớn hơn.

4.3. Kết quả giai đoạn 2 - Fine-tune EfficientNet-B0

4.3.1 Hiệu năng tổng thể

- Sau quá trình fine-tune chuyên sâu trên bộ dữ liệu 10 lớp [8], EfficientNet-B0 [4] đạt các chỉ số ấn tượng. Validation accuracy đạt 93.57% và test accuracy đạt 93.73%, cho thấy mô hình không bị overfitting. Macro F1-score đạt 0.92 và weighted F1-score đạt 0.94, phản ánh hiệu năng tốt và ổn định trên hầu hết các lớp. Kết quả này vượt xa so với giai đoạn 1, khẳng định hiệu quả của chiến lược fine-tune toàn bộ backbone kết hợp differential learning rate và class weighting.

Class	Precision	Recall	F1-score	Support
battery	1.00	1.00	1.00	76
cardboard	0.98	0.85	0.91	60
clothes	1.00	1.00	1.00	76
glass	0.95	0.93	0.94	76
metal	0.85	0.92	0.88	61
organic	1.00	0.97	0.99	77
paper	0.89	0.92	0.90	76
plastic	0.98	0.82	0.89	72
shoes	0.97	1.00	0.99	76
trash	0.55	0.90	0.68	20
Accuracy			0.94	670
Macro Avg	0.92	0.93	0.92	670
Weighted Avg	0.95	0.94	0.94	670

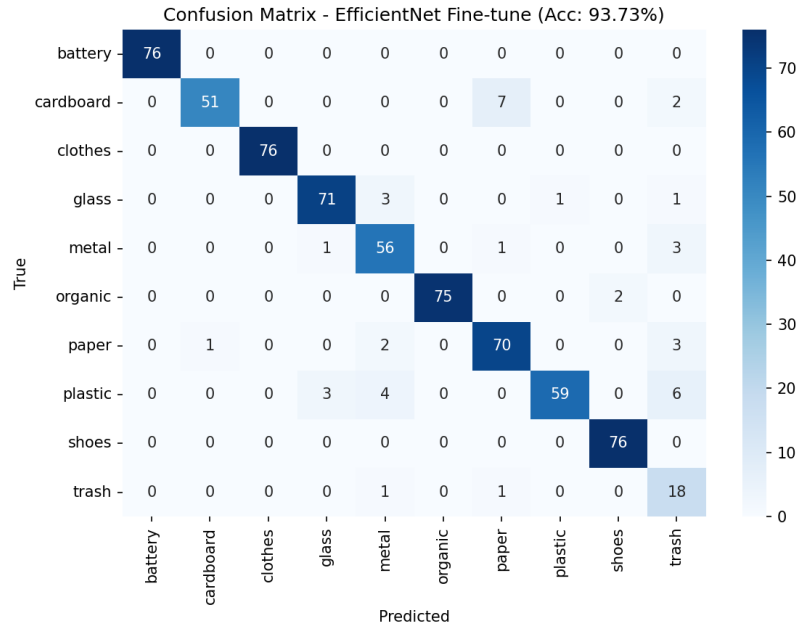
Bảng 5 - Kết quả đánh giá sau khi train EfficientNet-B0

4.3.2 Kết quả chi tiết theo từng lớp

- Phân tích chi tiết từng lớp cho thấy hầu hết các lớp đạt hiệu năng rất cao. Lớp battery đạt precision, recall và F1 đều bằng 1.00, tức phân loại hoàn hảo trên 76 mẫu test. Lớp clothes và shoes cũng đạt F1 lần lượt là 1.00 và 0.99. Lớp cardboard đạt F1 0.91, glass

0.94, organic 0.99, paper 0.90. Lớp plastic đạt F1 0.89 với recall 0.82. Lớp metal đạt F1 0.88 với precision 0.85 và recall 0.92.

- Lớp yếu nhất là trash với F1 chỉ đạt 0.68, precision 0.55 và recall 0.90. Recall cao (0.90) cho thấy mô hình nhận diện được phần lớn ảnh trash thực sự, nhưng precision thấp (0.55) cho thấy nhiều ảnh của lớp khác bị phân loại nhầm thành trash. Nguyên nhân trực tiếp là lớp trash chỉ có 137 ảnh - ít nhất trong 10 lớp - nên dù đã áp dụng class weight cao nhất (3.2531), mô hình vẫn thiếu dữ liệu đa dạng để học đặc trưng tốt cho lớp này.



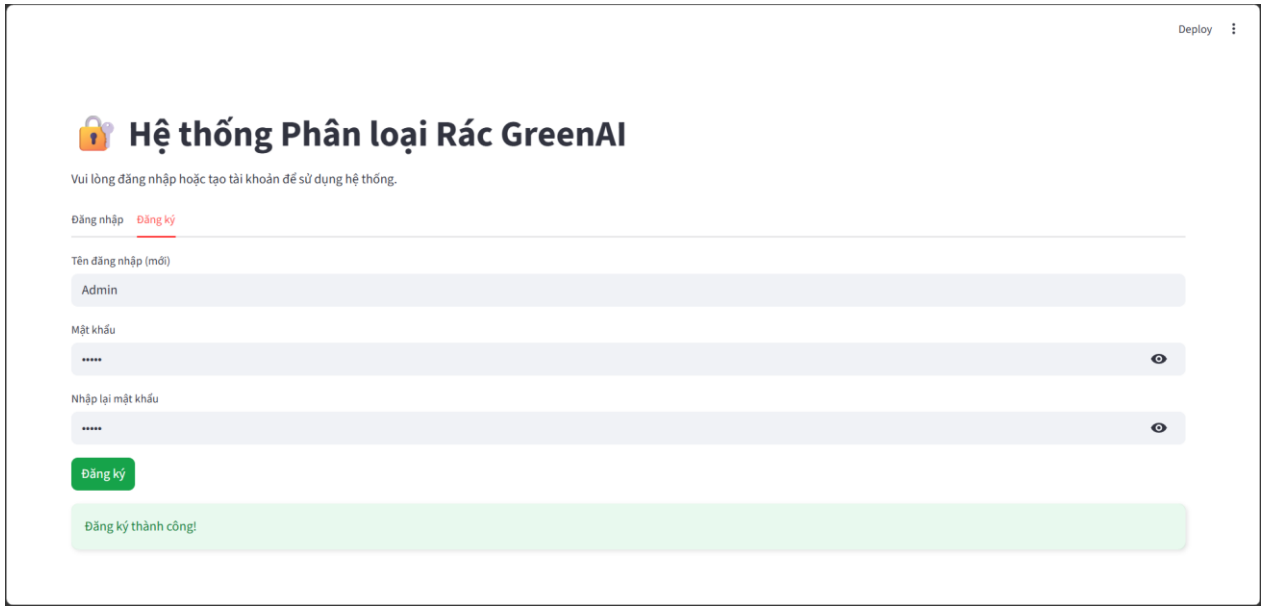
Hình 4 – Confusion Matrix – EfficientNet Fine-tune

4.3.3 Phân tích kết quả fine-tune

- Việc fine-tune toàn bộ backbone EfficientNet-B0 [4] với differential learning rate và Adam optimizer [15] chứng minh tính hiệu quả rõ rệt. Test accuracy tăng từ 79.21% (giai đoạn 1) lên 93.73% (giai đoạn 2), tức tăng hơn 14 điểm phần trăm. Kết quả này phù hợp với lý thuyết transfer learning: khi bài toán đích khác đáng kể so với nguồn, việc cho phép toàn bộ backbone thích nghi sẽ mang lại cải thiện lớn hơn so với chỉ học lại lớp classifier.

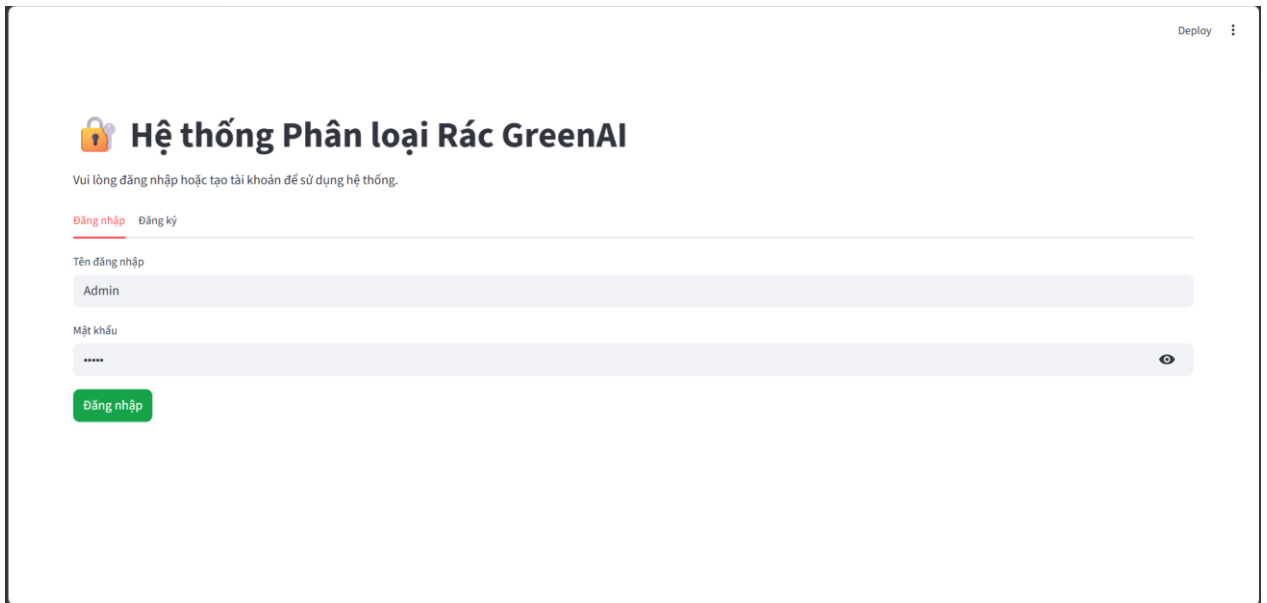
4.4. Kết quả triển khai ứng dụng

4.4.1 Chức năng đăng ký và đăng nhập



The screenshot shows the registration page of the 'Hệ thống Phân loại Rác GreenAI'. At the top right is a 'Deploy' button with a dropdown arrow. Below the title is a subtitle: 'Vui lòng đăng nhập hoặc tạo tài khoản để sử dụng hệ thống.' There are two tabs: 'Đăng nhập' (highlighted in red) and 'Đăng ký'. The 'Đăng ký' tab contains a form with three input fields: 'Tên đăng nhập (mới)' with the value 'Admin', 'Mật khẩu' with masked characters '*****', and 'Nhập lại mật khẩu' also with '*****'. Each field has an eye icon to toggle visibility. A green 'Đăng ký' button is at the bottom of the form. A green success message 'Đăng ký thành công!' is displayed below the button.

Hình 5 - Giao diện đăng ký thành công tài khoản

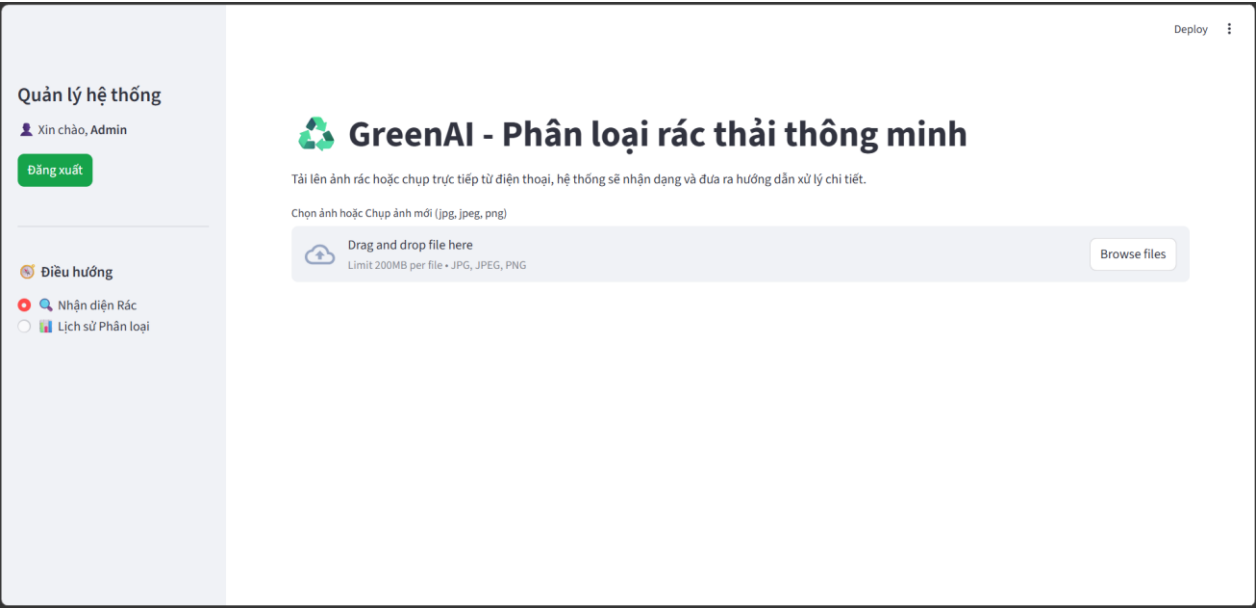


The screenshot shows the login page of the 'Hệ thống Phân loại Rác GreenAI'. It has the same header and subtitle as the registration page. The 'Đăng nhập' tab is highlighted in red, and the 'Đăng ký' tab is also visible. The form has two input fields: 'Tên đăng nhập' with the value 'Admin' and 'Mật khẩu' with masked characters '*****'. A green 'Đăng nhập' button is at the bottom of the form.

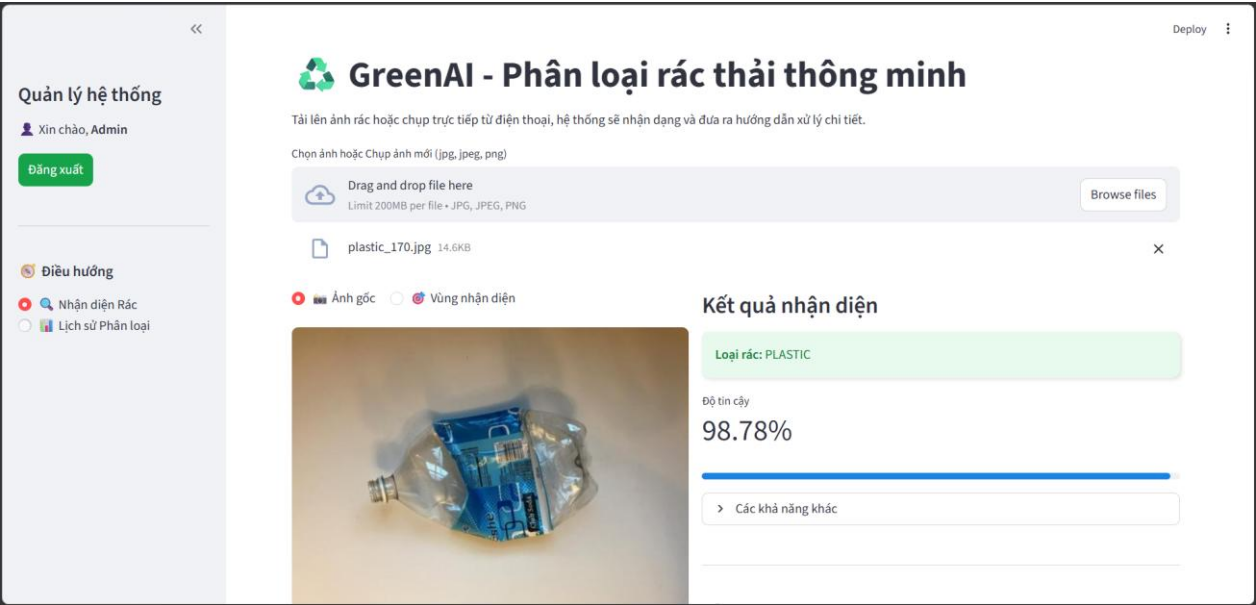
Hình 6 - Giao diện đăng nhập

- Giao diện xác thực người dùng gồm hai tab: đăng nhập cho người dùng đã có tài khoản và đăng ký cho người dùng mới. Sau khi đăng nhập thành công, thông tin phiên được lưu vào session state của Streamlit, cho phép người dùng điều hướng giữa các trang Scanner và Dashboard mà không cần đăng nhập lại. Toàn bộ lịch sử phân loại và dữ liệu phản hồi được gắn với tài khoản đăng nhập, đảm bảo tính riêng tư và phục vụ cho việc phân tích dữ liệu theo từng người dùng trong tương lai.

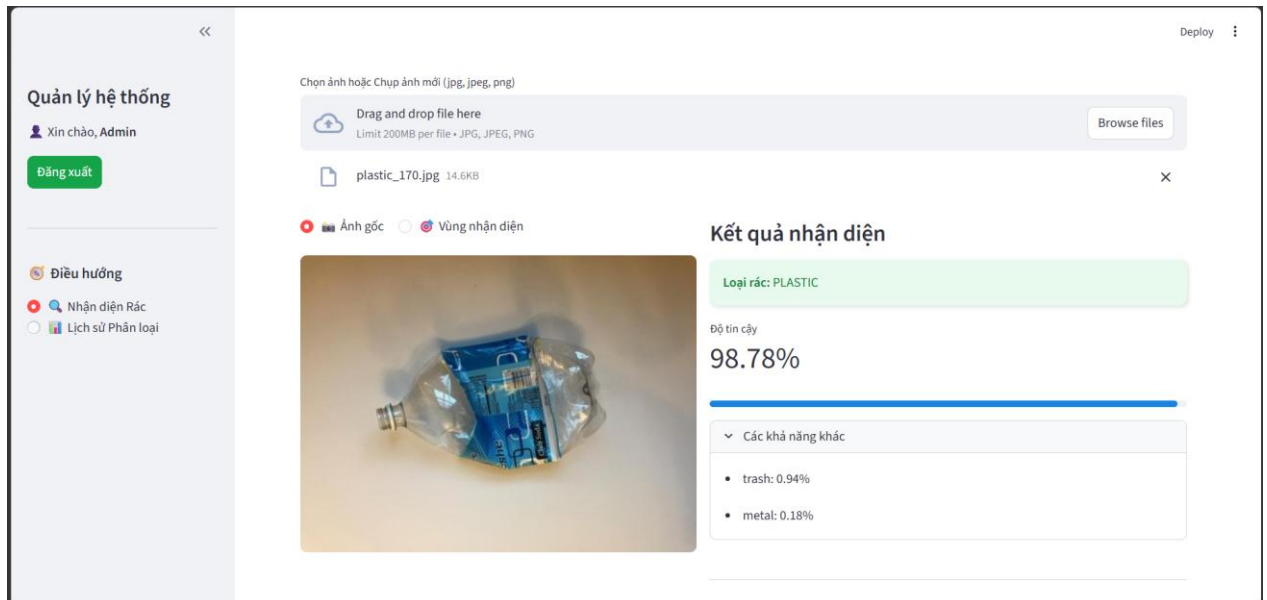
4.4.2 Chức năng phân loại



Hình 7 - Giao diện phân loại (1)



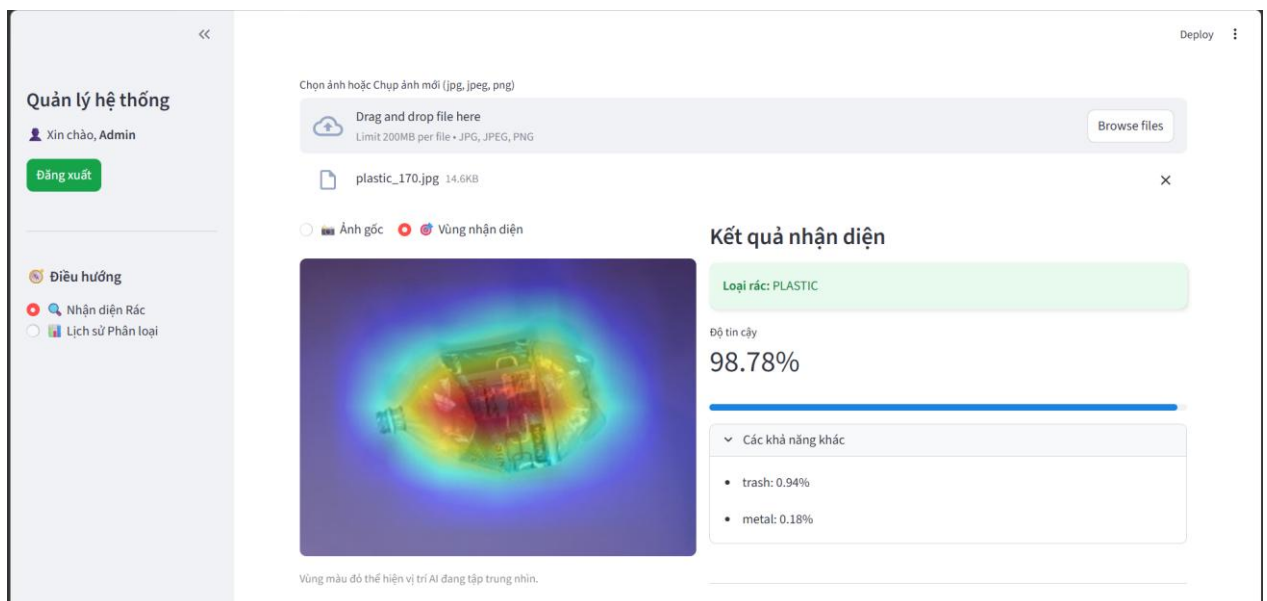
Hình 8 - Giao diện phân loại (2)



Hình 9 - Giao diện phân loại (3)

- Giao diện phân loại cho phép người dùng tải ảnh lên và nhận kết quả trong thời gian ngắn nhờ ONNX Runtime [11]. Kết quả hiển thị gồm nhãn lớp dự đoán hàng đầu, thanh tiến trình confidence cho top-3 lớp có xác suất cao nhất, và cảnh báo nếu confidence thấp hơn 60%.

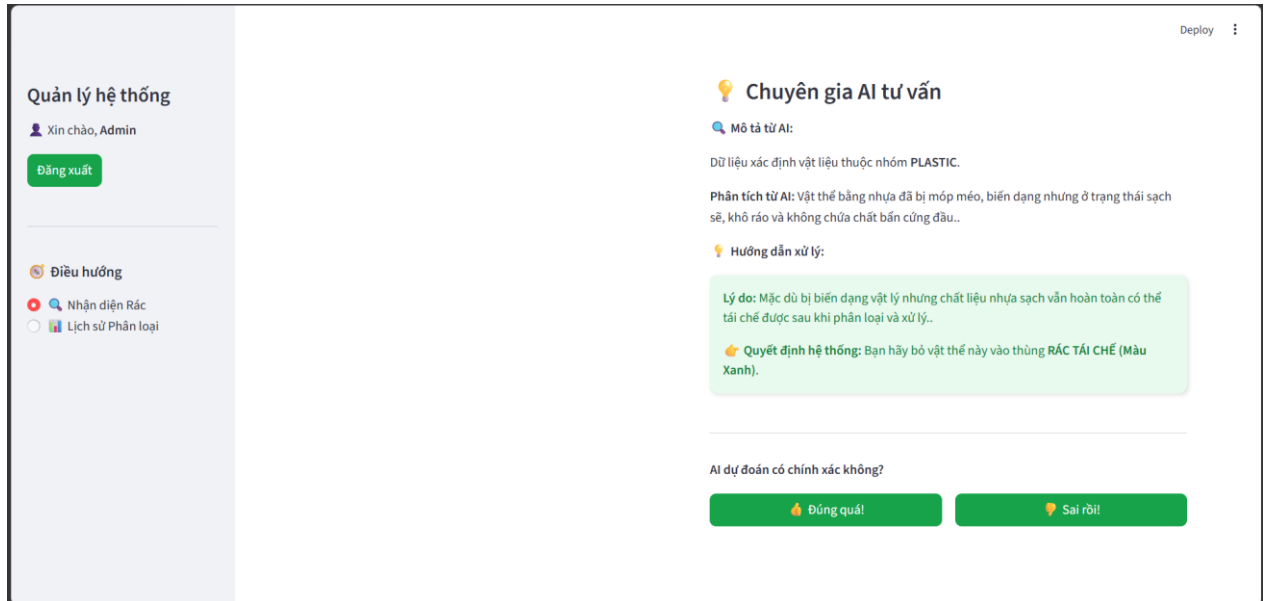
4.4.3 Hiển thị Grad-CAM



Hình 10 - Giao diện phân loại (Grad-CAM)

- Tính năng Grad-CAM [5] cho phép người dùng chuyển đổi giữa ảnh gốc và bản đồ nhiệt để thấy mô hình đang tập trung vào vùng nào. Các vùng màu đỏ đậm trên heatmap là những vùng có ảnh hưởng lớn nhất đến quyết định phân loại. Tính năng này giúp người dùng tin tưởng hơn vào kết quả khi thấy mô hình chú ý đúng vào đối tượng rác, và phát hiện những trường hợp mô hình đưa ra quyết định dựa trên nền ảnh thay vì đối tượng chính.

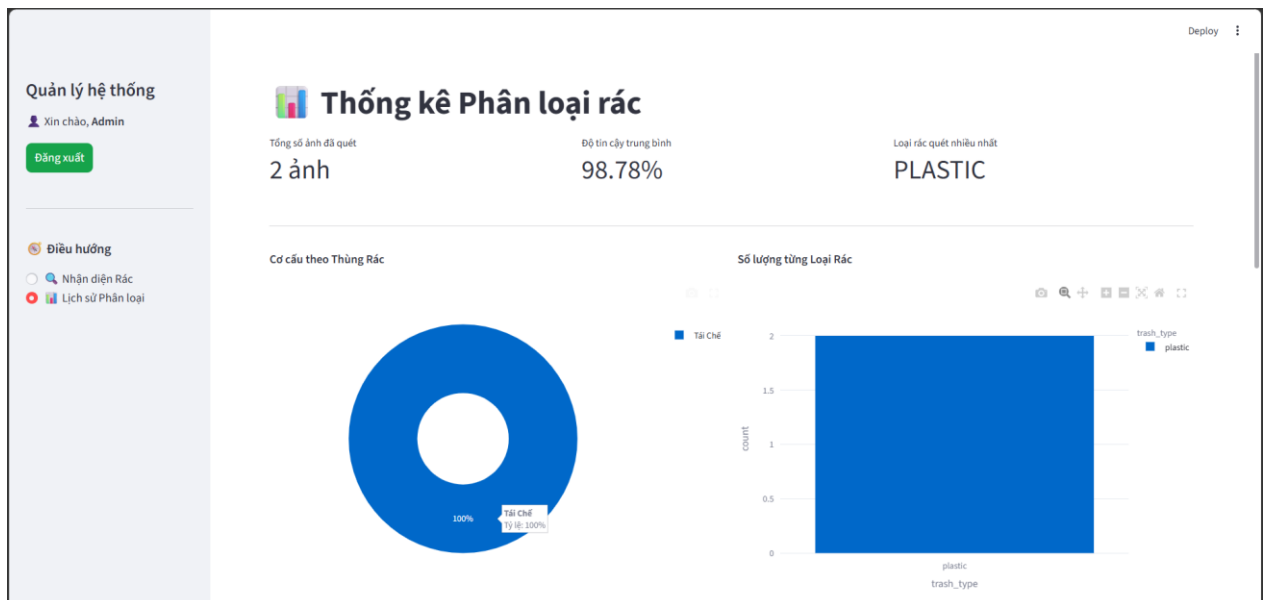
4.4.4 Tư vấn thông minh Gemini



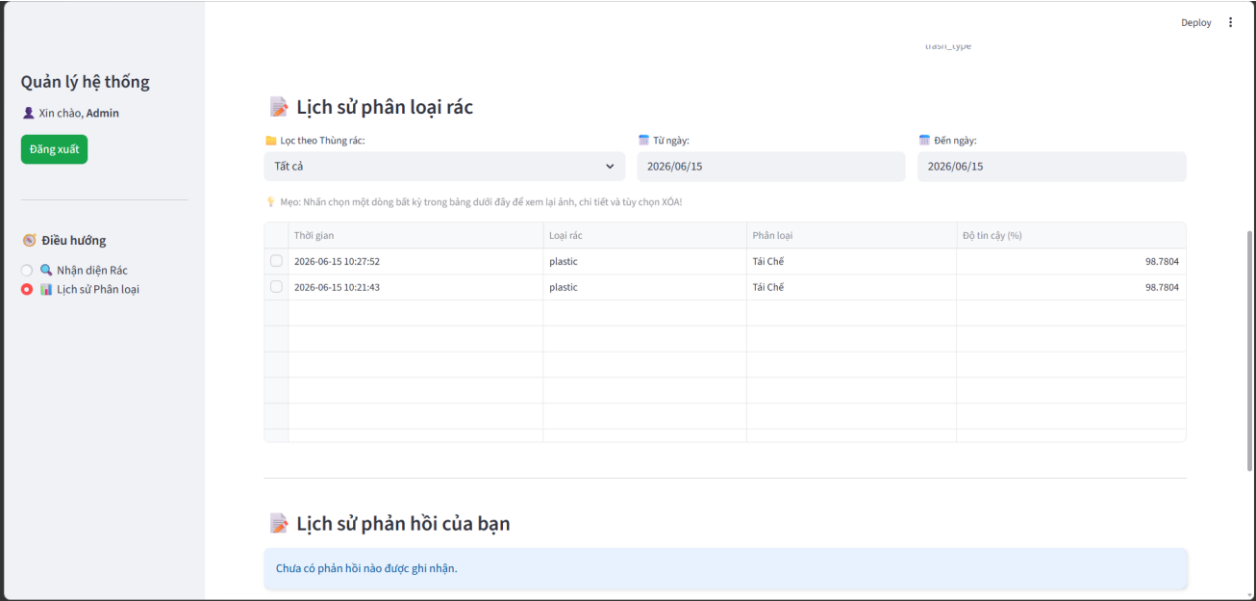
Hình 11 - Giao diện phân loại (Tư vấn thông minh Gemini)

- Sau khi phân loại, Gemini Vision [12] phân tích tình trạng vật thể và đưa ra lời khuyên cụ thể về cách xử lý. Hệ thống phân biệt ba trạng thái: recyclable, non_recyclable và hazardous. Lời khuyên được tổng hợp từ cả nhãn lớp phân loại và đánh giá tình trạng của Gemini, cho kết quả tư vấn phù hợp với ngữ cảnh thực tế hơn.

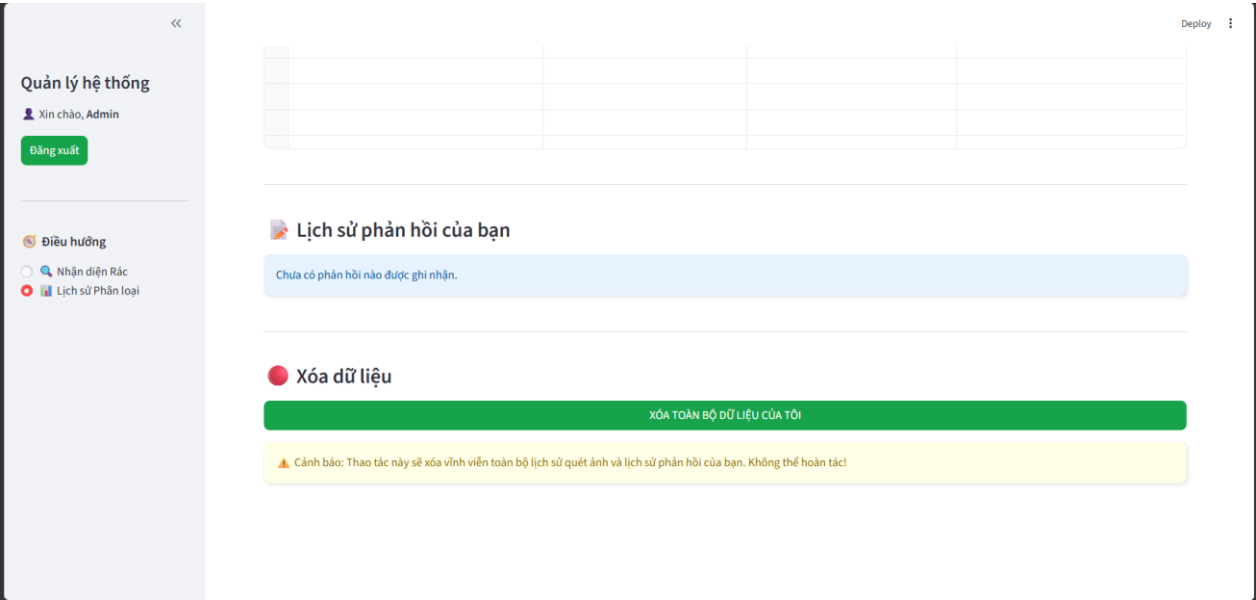
4.4.5 Dashboard thống kê



Hình 12 - Giao diện Dashboard thống kê (1)



Hình 13 - Giao diện Dashboard thống kê (2)



Hình 14 - Giao diện Dashboard thống kê (3)

- Trang dashboard hiển thị thống kê lịch sử phân loại của người dùng, bao gồm biểu đồ phân bố các loại rác đã scan, timeline theo thời gian và bộ lọc theo ngày.

4.4.6 Đánh giá tổng hợp

- Hệ thống GreenAI đáp ứng tốt các yêu cầu đặt ra ban đầu. Về độ chính xác, test accuracy 93.73% vượt mục tiêu 90%. Về tính giải thích, Grad-CAM [5] cung cấp trực quan hóa rõ ràng và dễ hiểu. Về độ tin cậy, cơ chế fallback Gemini và xử lý lỗi Grad-CAM đảm bảo hệ thống không crash trong các tình huống bất thường. Về bảo mật, mật khẩu được mã hóa bcrypt và lịch sử được lưu trữ có tổ chức. Điểm yếu duy nhất được ghi nhận là hiệu năng trên lớp trash còn hạn chế do thiếu dữ liệu.

CHƯƠNG V. KẾT LUẬN

5.1. Tổng kết kết quả đạt được

- Đề tài đã hoàn thành đầy đủ ba mục tiêu đặt ra ban đầu:

- Về nghiên cứu và so sánh mô hình, ba kiến trúc VGG16 [2], ResNet50 [3] và EfficientNet-B0 [4] đã được huấn luyện và đánh giá một cách có hệ thống trên bộ dữ liệu TrashNet 6 lớp [7]. Kết quả so sánh cung cấp góc nhìn thực tiễn về sự đánh đổi giữa độ chính xác, kích thước mô hình và tốc độ huấn luyện.
- Về fine-tune mô hình, EfficientNet-B0 [4] sau khi được fine-tune chuyên sâu trên bộ dữ liệu 10 lớp [8] đã đạt test accuracy 93.73% và macro F1-score 0.92. Việc áp dụng kết hợp unfreeze toàn bộ backbone, differential learning rate, Adam optimizer [15], class weighting và data augmentation đã chứng minh hiệu quả rõ rệt.
- Về triển khai ứng dụng, hệ thống GreenAI hoàn chỉnh đã được xây dựng với đầy đủ các tính năng: phân loại ảnh real-time bằng ONNX Runtime [11], giải thích AI bằng Grad-CAM [5], tư vấn thông minh bằng Gemini Vision [12], quản lý tài khoản người dùng, lưu trữ lịch sử và dashboard thống kê.

5.2. Hạn chế

- Hạn chế lớn nhất là hiệu năng thấp trên lớp trash với F1-score chỉ đạt 0.68, xuất phát từ việc lớp này chỉ có 137 mẫu huấn luyện trong bộ dữ liệu [8]. Class weighting đã giúp cải thiện recall nhưng chưa đủ để nâng precision do thiếu tính đa dạng trong dữ liệu huấn luyện.

- Hạn chế thứ hai là bộ dữ liệu Kaggle [8] được chụp trong điều kiện tương đối lý tưởng, ảnh rõ ràng và đối tượng rác rõ nét. Trong thực tế, rác thải thường xuất hiện trong điều kiện phức tạp hơn, làm giảm hiệu năng của mô hình.

- Hạn chế thứ ba là hệ thống hiện chưa có cơ chế phát hiện ảnh không phải rác, khiến mô hình vẫn cưỡng bức phân loại ngay cả khi ảnh đầu vào không liên quan đến rác thải.

5.3. Hướng phát triển

- Hướng thứ nhất là tăng cường dữ liệu cho lớp trash thông qua thu thập thêm ảnh thực tế và áp dụng các kỹ thuật augmentation chuyên biệt.

- Hướng thứ hai là triển khai hệ thống Active Learning dựa trên dữ liệu feedback đã thu thập. Khi người dùng cung cấp đủ phản hồi, hệ thống có thể tự động đề xuất các mẫu cần gán nhãn và retrain định kỳ để mô hình liên tục cải thiện theo thời gian.

- Hướng thứ ba là xây dựng cơ chế phát hiện ảnh ngoài phạm vi (out-of-distribution detection), cho phép hệ thống từ chối phân loại khi ảnh đầu vào không phải rác thải, thay vì cưỡng bức đưa ra kết quả không đáng tin cậy. Hiện tại hệ thống đã có cảnh báo khi confidence thấp hơn 60%, nhưng chưa có cơ chế nhận biết ảnh hoàn toàn không liên quan đến rác.

- Hướng thứ tư là mở rộng sang nhận dạng thời gian thực qua camera, tích hợp với các hệ thống IoT như thùng rác thông minh, và xây dựng ứng dụng di động nhẹ bằng cách chạy ONNX model [11] trực tiếp trên thiết bị.

5.4. Kết luận chung

- Đề tài đã thành công trong việc xây dựng một hệ thống phân loại rác thải thông minh hoàn chỉnh, kết hợp các kỹ thuật học sâu hiện đại [1] với khả năng giải thích AI [5] và tư vấn thông minh [12]. Kết quả đạt được - test accuracy 93.73% với mô hình chỉ nặng 15.6 MB - cho thấy EfficientNet-B0 [4] là lựa chọn tối ưu cho bài toán phân loại rác thải trên thiết bị có tài nguyên hạn chế. Hệ thống không chỉ đáp ứng yêu cầu kỹ thuật mà còn hướng đến ứng dụng thực tiễn, đặt nền móng cho các hướng phát triển tiếp theo trong lĩnh vực AI ứng dụng cho môi trường.

TÀI LIỆU THAM KHẢO

- [1] Y. LeCun, Y. Bengio, và G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [2] K. Simonyan và A. Zisserman, "Very deep convolutional networks for large-scale image recognition," trong *Proc. Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [3] K. He, X. Zhang, S. Ren, và J. Sun, "Deep residual learning for image recognition," trong *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.
- [4] M. Tan và Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," trong *Proc. Int. Conf. Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.
- [5] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, và D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," trong *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 618–626.
- [6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, và L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," trong *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Miami, FL, USA, 2009, pp. 248–255.
- [7] G. Thung và M. Yang, "Classification of trash for recyclability status," CS229 Project Report, Stanford University, Stanford, CA, USA, 2016. [Online]. Available: <https://github.com/garythung/trashnet>
- [8] N. Kansara, "Garbage Classification Dataset," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/nishchalkansara/garbage-classification>
- [9] S. Ramírez, "FastAPI," 2019. [Online]. Available: <https://fastapi.tiangolo.com>
- [10] Streamlit Inc., "Streamlit - The fastest way to build data apps," 2019. [Online]. Available: <https://streamlit.io>
- [11] Microsoft và cộng sự, "ONNX Runtime: Cross-platform, high performance ML inferencing," 2018. [Online]. Available: <https://onnxruntime.ai>
- [12] Google DeepMind, "Gemini: A family of highly capable multimodal models," arXiv preprint arXiv:2312.11805, 2023.
- [13] A. Paszke và cộng sự, "PyTorch: An imperative style, high-performance deep learning library," trong *Advances in Neural Information Processing Systems (NeurIPS)*, Vancouver, Canada, 2019, pp. 8024–8035.
- [14] F. Pedregosa và cộng sự, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [15] D. P. Kingma và J. Ba, "Adam: A method for stochastic optimization," trong *Proc. Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.